

CSC 240 Text Processing Project: Stage 1 Plan

Conor Burns & Patrick Clisham | March 27, 2026

1. Requirements

Read the CSV file and store each email as an object with its text and label.
Extract features from each email's text (word count, char count, spam word count, etc.).
Display features for a single email.
Compute and display a summary (average) of features across a group of emails.
Save email features to email_features.csv.
Save group summary to summary_features.csv.
Compute distance between two emails based on their features.
Build a spam model and not-spam model from training data.
Classify a new email and write predictions to predictions.txt.

2. Class Design & UML

Two classes: Email stores a single email and its features. Dataset holds a list of Email objects and handles loading, summarizing, and classifying.

Public class Email	Public class Dataset
- String text	- ArrayList<Email> entries
- int label	
- int wordCount	
- int urlCount	
- int numberCount	+ loadCSV(String filename)
- double avgWordLength	+ addEntry(Email e)
	+ getSpamEntries()
	+ getNotSpamEntries()
+ getWordCount()	+ getSummary()
+ getUrlCount()	+ saveFeatures(String filename)
+ getNumberCount()	+ saveSummary(String filename)
+ getAvgWordLength()	+ classify(Email e)
+ displayFeatures()	+ savePredictions(String filename)
+ distance(Email other)	

3. Planning Questions

Q1. How will you read the CSV?

Use `BufferedReader` to read line by line. Split each row on commas to get the email text and label.

Q2. Which features will you compute?

- Word count - total number of words in the email
- Char count - total number of characters
- URL count - number of URL tokens in the text
- NUMBER count - number of NUMBER tokens in the text// doesn't count
- Average word length
- Spam words - can't hardcode that, research alternative methods instead of hardcoding// get all words in spam but not ham and use that as the list of spam words

Q3. How will you represent an email in code?

An `Email` class with the raw text, label (1/0/-1), and fields for each computed feature.

Q4. How will you save features and summaries?

Write to CSV using `PrintWriter`. `email_features.csv` has one row per email.

`summary_features.csv` has the average of each feature across spam and not-spam groups.

Q5. How will you display features and summaries?

`displayFeatures()` prints each feature and its value. `getSummary()` prints the average per feature for a group.

Q6. What distance metric will you use?

Euclidean distance - $\sqrt{\text{sum of squared differences between each feature}}$. Missing values treated as 0.

Q7. How will you compute distance from an email to a group?

Compute the average feature values across the group. Then use the same distance formula comparing the email to that average. Closest group is the predicted label.

4. Testing Plan

One test per requirement:

- T1 (R1) - Load a small test CSV. Check that `Email` objects are created with correct text and label.
- T2 (R2) - Run feature extraction on a known string. Check word count, url count, and number count.
- T3 (R3) - Call `displayFeatures()` and check that output contains expected feature names.
- T4 (R4) - Build a `Dataset` from a few emails and check `getSummary()` returns correct averages.
- T5 (R5) - Save features for 2 emails to CSV. Read it back and check row count and headers.
- T6 (R6) - Save summary to CSV. Read it back and check values match expected averages.

- T7 (R7) - Two emails with known features. Check distance() returns the right value.
- T8 (R8) - Train on a small labeled set. Check spam and not-spam groups have correct sizes.
- T9 (R9) - Classify an email that clearly looks like spam. Check predictions.txt output.

5. Schedule & Workload

- 3/27–3/30 [Burns] - Email class - fields, feature extraction methods, displayFeatures(). Write T1, T2, T3.
- 3/27–3/30 [Clisham] - Dataset class - loadCSV(), addEntry(), getSpamEntries(), getNotSpamEntries(). Write T4.
- 3/31–4/4 [Burns] - saveFeatures() and saveSummary() to CSV. Write T5, T6.
- 3/31–4/4 [Clisham] - distance() method and getSummary(). Write T7.
- 4/7–4/11 [Both] - classify() and savePredictions(). Train/test split. Write T8, T9.
- 4/14–4/25 [Both] - Testing, accuracy checks, and cleanup.
- 4/28–5/1 [Both] - Final submission.

The average ammount of times a certain word occurs in a spam email and ham email
 The 5 spam 11 ham if 8 then 5 words 2 go to ham and to spam then it is automatically spam
google.com